

What Is a Cost-Based Optimizer?

Last Updated September 17, 2021

[Brent Ozar](#)

[Execution Plans](#)

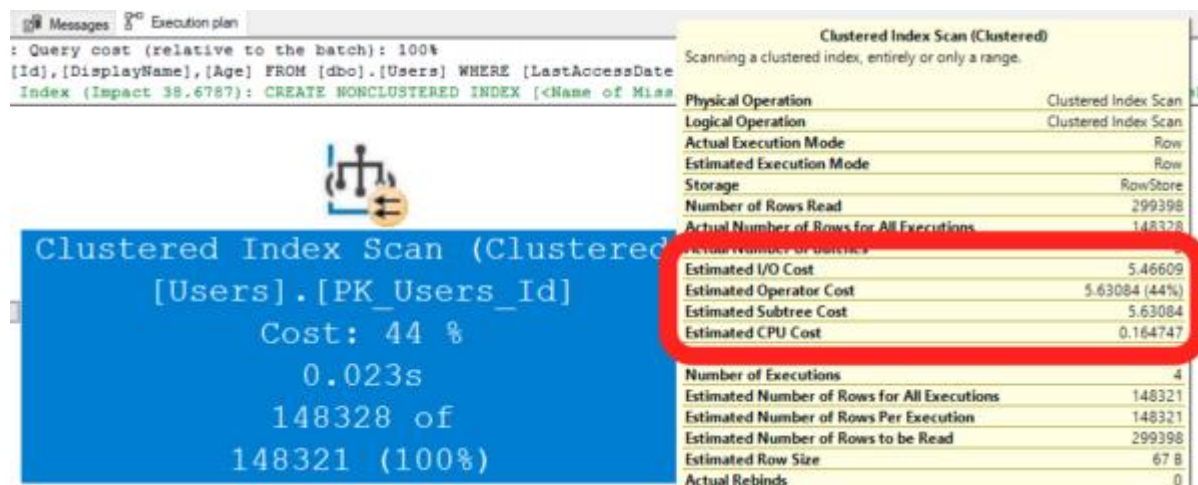
0

When you execute a query, the database server has to figure out things like:

- Which table to process first
- Which index to use on that table
- Whether to seek on that index or scan it
- Which table to process next
- How to join the data between those two tables
- When to sort the data

For even the simplest queries, there are usually several possible ways to get the job done. The database server has to figure out which way is going to be the fastest – or at least, fast enough for the meatbag who ran it.

One way to do that is to build a cost-based query optimizer. As it builds execution plans, it assigns a cost to each operation inside the execution plan. That's how SQL Server does it, and you can see the evidence if you hover your mouse over components in an execution plan:



Clustered Index Scan (Clustered)

Scanning a clustered index, entirely or only a range.

Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	299398
Actual Number of Rows for All Executions	148328
Estimated I/O Cost	5.46609
Estimated Operator Cost	5.63084 (44%)
Estimated Subtree Cost	5.63084
Estimated CPU Cost	0.164747
Number of Executions	4
Estimated Number of Rows for All Executions	148321
Estimated Number of Rows Per Execution	148321
Estimated Number of Rows to be Read	299398
Estimated Row Size	67.8
Actual Rebinds	0

Clustered Index Scan (Clustered)

[Users].[PK_Users_Id]

Cost: 44 %

0.023s

148328 of

148321 (100%)

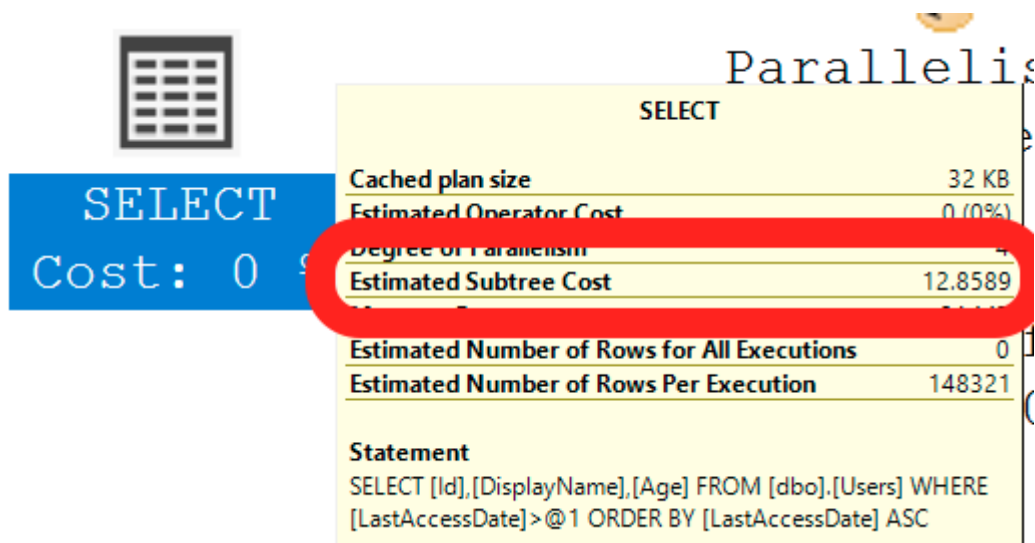
In this case, I'm hovering my mouse over the Clustered Index Scan, and you see a few cost details.

- Estimated I/O Cost 5.46609 – means that SQL Server thinks this operation will cost 5.46609 units. (More on the units in a second.) The larger the table is, the higher this cost is going to be. SQL Server even makes guesses about how much of the data will be in cache, versus how much will need to be read from storage.
- Estimated CPU Cost 0.165757 – some operations require a lot of CPU work, but not this one. Here, we're just scanning a table, kinda like you scanning a phone book. You're not really going to be thinking hard – you're just looking for a particular last

name. You'd probably get bored quickly. (Maybe not, though, because you're easily amused.)

- Estimated Operator Cost 5.63084 – add up the two above numbers, and you get this one. Yes, usually when you have two numbers that make up a total, you'd put the two small ones first, then the total next. No, that's not how SQL Server does it. When SQL Server shows data in a tooltip, it's either in alphabetical order, or they load up the Data Cannon™ and fire it at the screen. Wherever the numbers end up, that's where they end up. This tooltip was rendered with the Data Cannon™.

If you add up the costs of all of the operators in a query plan, you get its total cost, as shown here by hovering my mouse over the SELECT's operator itself:



In this case, this particular execution plan costs 12.8589. (Again – we'll get back to what the unit of measure is here in a second.)

SQL Server builds multiple plans, and uses costs to compare them.

After building the first plan, SQL Server rolls up its sleeves and says, "Alright, let's give 'er another shot." It builds another execution plan, and then compares that new plan's cost to the cost of the original plan. After a few iterations of this, it keeps whichever plan is the least expensive cost, and runs that.

I'm simplifying a lot here:

- Sometimes a query is trivially simple, so SQL Server only builds one plan.
- Sometimes SQL Server builds a plan or two, and then says, "Screw it, these costs are so inexpensive, it doesn't make sense to keep building plans. I should just roll with this plan because it's good enough."
- Sometimes SQL Server builds a plan and says, "Sweet Potato, would you look at the time! If I tried to build another execution plan, it might take way too much time. I'm just gonna call time here, and we're going to run with this plan, even though I might be able to build a better plan given more time."

A cost-based optimizer is about using costs of query plans in order to determine which query plan to run with. It isn't necessarily about exhaustively trying every possible query plan, or...buckle up...even about having accurate costs.

Cost-based optimizer implementations aren't necessarily perfect.



Paul White makes big Query Bucks

The costs are arbitrary. The database system has to decide how to assign costs to execution plan operators. You'd love for them to use "estimated time," but the reality is that – well, go ask your project managers how accurate their estimates have been lately. It's really hard to guess times. Instead, SQL Server guesses how much CPU and IO work will be required. A long time ago, this used to be guessed in seconds, but these days, we call the unit of measure [Query Bucks](#).

The costs don't reflect your hardware. The cost-based optimizer was built in the 1990s, and it hasn't been updated to reflect modern CPU, memory, or storage speeds. We're still using the same costs per key lookup that we used over twenty years ago. Personally, I don't think Microsoft should even *try* to make costs reflect current hardware: after all, when a SQL Server gets installed, it might be used for ten years or more across all kinds of different hardware. It'd be hopeless to try to make the costs accurate for hardware that hasn't even been built yet.

Some query work doesn't get a cost. For example, memory grants aren't taken into account here: you can see identical queries with a 1MB and a 100GB memory grant that have exactly the same cost.

Even things with a cost aren't necessarily accurate. Some operations like multi-statement table-valued functions and linked server queries can't be estimated accurately. Or maybe they *could* be, but Microsoft just never took the time to build accurate estimations into the cost-based optimizer. As a result, their work might be underestimated or overestimated, leading to inefficient query plans.

The more you know, the faster you go.

SQL Server's cost-based optimizer is more than good enough to handle the vast majority of scenarios. Most query authors just never need to know how the cost-based optimizer works, let alone its gotchas and pitfalls. They just write queries, and the queries run well enough – especially given how small many databases are.

However, as your data grows and your query complexity grows, the more you start asking questions about why your query isn't running as quickly as you might expect. That's when you start peeling back the covers on your query plans, trying to use query hints to boss the optimizer around into making different decisions.

That's your sign that you need training on how the cost-based optimizer works. Start with [my totally free How to Think Like the Engine class](#), and go from there.

And hey, it just so happens that [I'm teaching it live for free next month](#).